

TypeORM — Installation & Configuration

Section	10 — TypeORM Introduction & Setup
Subtopic	10-02 — Installation & Configuration
Duration	~2 hours (part of the 6–8 hr section)
Tags	Theory Practical TypeScript
Topics covered	Installing TypeORM and drivers, tsconfig decorator flags, DataSource object, inline vs file-based config, environment variables with dotenv, verifying the connection, logging

Learning Objectives — This Subtopic

- Install TypeORM and the correct database driver for the target database
- Configure `tsconfig.json` with the two decorator flags TypeORM requires
- Construct a `DataSource` object with all required options and initialise it
- Externalise database credentials using environment variables and `dotenv`
- Verify a working database connection and interpret TypeORM's startup log output
- Explain why `synchronize: true` is dangerous in production and what to use instead

What You Are Installing

TypeORM is not a standalone package — it always works alongside a database-specific **driver**. The driver handles the low-level connection, authentication, and wire protocol for the target database. TypeORM generates SQL and delegates its execution to the driver. You must install both.

Package	Role	Required for
---------	------	--------------

<code>typeorm</code>	The ORM itself — entities, repositories, QueryBuilder, migrations	Always
<code>reflect-metadata</code>	Polyfill that enables TypeScript decorator metadata reflection at runtime	Always — TypeORM cannot read decorator metadata without it
<code>better-sqlite3</code>	SQLite driver — file-based database, no server required	SQLite projects (practicals)
<code>pg</code>	PostgreSQL driver	PostgreSQL projects
<code>mysql2</code>	MySQL / MariaDB driver	MySQL projects
<code>dotenv</code>	Loads <code>.env</code> files into <code>process.env</code>	Any project using environment variables for config

Installation

For this module's practicals (SQLite):

```
npm install typeorm reflect-metadata better-sqlite3 dotenv
```

```
npm install --save-dev @types/better-sqlite3
```

For a PostgreSQL production project, replace the driver:

```
npm install typeorm reflect-metadata pg dotenv
```

```
npm install --save-dev @types/pg
```

Note — `reflect-metadata` : This package must be imported exactly once, at the very top of your application's entry point, before any TypeORM entity is imported. If it is imported too late — after entity files have been loaded — decorator metadata will not be registered and TypeORM will throw cryptic errors about missing column metadata. The import is a side-effect import: `import 'reflect-metadata';` with no named imports.

TypeScript Configuration for TypeORM

TypeORM's decorator-based API requires two compiler options that are not in a standard `tsconfig.json`. Without them, TypeScript compiles but TypeORM silently fails to register entity metadata at runtime, producing confusing errors when you attempt your first query.

Option	What it does	Required?
<code>"experimentalDecorators": true</code>	Enables the TypeScript decorator syntax (<code>@Entity()</code> , <code>@Column()</code> , etc.) — still a proposal, hence "experimental"	Yes — TypeORM will not compile without it
<code>"emitDecoratorMetadata": true</code>	Causes the TypeScript compiler to emit type metadata (the actual TypeScript type of each decorated property) into the compiled output — read by <code>reflect-metadata</code> at runtime	Yes — TypeORM uses this to infer column types automatically from your TypeScript types

Here is the complete `tsconfig.json` for a TypeORM project, extending the one established in Section 09-03:

`tsconfig.json`

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "lib": ["ES2020"],
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
  }
}
```

```
"resolveJsonModule": true,  
  
// — TypeORM-required options —  
"experimentalDecorators": true,  
"emitDecoratorMetadata": true  
},  
"include": ["src/**/*"],  
"exclude": ["node_modules", "dist"]  
}
```

Important — strict: true and nullable columns: With `strict: true` enabled, TypeScript requires all class properties to be initialised in the constructor, or marked as potentially undefined. TypeORM entity properties are initialised by the ORM, not a constructor — TypeScript does not know this. You will see errors like *"Property 'id' has no initialiser and is not definitely assigned in the constructor."* The standard solution is to use the definite assignment assertion (`!`): `id!: number;`. This tells TypeScript "I guarantee this will be assigned before use — trust me." You will see this throughout entity definitions in Section 11.

The DataSource Object

In TypeORM, the `DataSource` class is the central configuration and connection object. It holds every piece of information TypeORM needs to connect to a database and manage entities: the database type, connection credentials, the list of entity classes, and behavioural options like logging and schema synchronisation.

Prior to TypeORM 0.3 (released 2022), configuration was placed in a file called `ormconfig.json` which TypeORM read automatically. That approach is now deprecated. All modern TypeORM projects construct a `DataSource` instance directly in TypeScript code. This is the only approach covered in this module.

Note — `ormconfig.json` in older projects: If you open a legacy TypeORM codebase and see an `ormconfig.json` or `ormconfig.js` in the root, this is the pre-0.3 pattern. It still works in current TypeORM but is no longer documented and will likely be removed in a future major version. When you encounter it, treat the file's contents as equivalent to the options object passed to `new DataSource()`.

Stage 1 — Minimal DataSource (SQLite, hardcoded)

The minimum viable DataSource for SQLite. This establishes a working connection before any entities exist — useful for verifying the setup is correct.

First, the project structure at this stage:

```
typeorm-demo/
├─ src/
│   ├─ app.ts           ← entry point
│   └─ data-source.ts   ← DataSource instance lives here
├─ .env                 ← credentials (not committed to git)
├─ .gitignore
├─ package.json
└─ tsconfig.json
```

`src/data-source.ts`

```
import 'reflect-metadata';
import { DataSource } from 'typeorm';

export const AppDataSource = new DataSource({
  type: 'better-sqlite3', // the database engine
  database: 'database.db', // SQLite: path to the .db file (created automatically)
  synchronize: true,      // auto-create/alter tables to match entities on startup
  logging: true,          // print generated SQL to the console
  entities: [],           // entity classes will be added here in Section 11
  migrations: [],        // migration files – covered in Section 12
});
```

`src/app.ts`

```
import 'reflect-metadata';
import { AppDataSource } from './data-source';

AppDataSource.initialize()
  .then(() => {
    console.log('Data Source has been initialised.');
```

Run it:

```
npm run dev
```

Output:

```
query: BEGIN TRANSACTION
query: COMMIT
Data Source has been initialised.
```

TypeORM opens the SQLite file (creating it if it does not exist), runs an internal setup transaction, and resolves the promise. The `database.db` file now exists in your project root. With `entities: []`, no tables are created yet — that changes in Section 11 when entity classes are registered.

What each DataSource option means:

Option	Type	Description
<code>type</code>	<code>string</code>	The database engine identifier. Must match the installed driver package. Common values: <code>'better-sqlite3'</code> , <code>'postgres'</code> , <code>'mysql'</code> .
<code>database</code>	<code>string</code>	For SQLite: file path to the <code>.db</code> file. For PostgreSQL/MySQL: the database name.

<code>synchronize</code>	<code>boolean</code>	When <code>true</code> , TypeORM compares entity definitions against the live database schema on startup and issues <code>ALTER TABLE</code> / <code>CREATE TABLE</code> statements to make them match. See warning below.
<code>logging</code>	<code>boolean string[]</code>	When <code>true</code> , all generated SQL is printed to the console. Accepts an array for selective logging: <code>['query', 'error', 'warn']</code> .
<code>entities</code>	<code>Function[] string[]</code>	The entity classes TypeORM should manage. Can be class references or glob patterns like <code>'src/entity/**/*.ts'</code> .
<code>migrations</code>	<code>Function[] string[]</code>	Migration classes or file glob patterns. Covered in Section 12.

Important — `synchronize: true` in production: Schema synchronisation runs `ALTER TABLE` and `DROP COLUMN` statements automatically on every application startup. In production, this can silently destroy data: rename a property in your entity and the old column is dropped, taking its data with it. `synchronize: true` is acceptable during early development against a database you can afford to reset. As soon as you have data that matters — even in staging — switch to `synchronize: false` and use TypeORM **migrations** (Section 12) to evolve the schema safely and explicitly.

Environment Variables and dotenv

Database credentials — host, port, username, password, database name — must never be hardcoded in source files that are committed to version control. The standard approach is environment variables: values that exist in the process's environment rather than in the code itself. Different values are set in development, staging, and production without changing a single line of code.

Analogy — Think of environment variables as a configuration panel outside the machine. The machine (your application) is built once and deployed anywhere. The panel on the wall of each environment — development, staging, production — has different switches set. The machine reads the panel when it starts up and behaves accordingly. Hardcoding credentials is equivalent to wiring the configuration directly into the machine's circuit board: you need a different machine for each environment.

The .env File

`dotenv` reads a file named `.env` in the project root and loads its key-value pairs into `process.env`. The file is written in a simple format — one assignment per line.

`.env`

```
DB_TYPE=better-sqlite3
DB_DATABASE=database.db
DB_SYNCHRONIZE=true
DB_LOGGING=true
```

Important — Never commit `.env` to git. Add it to `.gitignore` immediately when the file is created. Commit a `.env.example` file instead — identical structure, but with placeholder values — so other developers know which variables to set without exposing real credentials.

`.gitignore`


```
node_modules/  
dist/  
*.db  
.env
```

`.env.example`

```
DB_TYPE=better-sqlite3  
DB_DATABASE=database.db  
DB_SYNCHRONIZE=true  
DB_LOGGING=true
```

Stage 2 — DataSource Configured from Environment Variables

Call `dotenv.config()` before constructing the DataSource. This populates `process.env` from `.env`, making the values available to read.

`src/data-source.ts`

```
import 'reflect-metadata';  
import { DataSource } from 'typeorm';  
import * as dotenv from 'dotenv';  
  
dotenv.config(); // loads .env into process.env – must run before DataSource is constructed  
  
export const AppDataSource = new DataSource({  
  type: process.env.DB_TYPE as 'better-sqlite3',  
  database: process.env.DB_DATABASE ?? 'database.db',  
  synchronize: process.env.DB_SYNCHRONIZE === 'true',  
  logging: process.env.DB_LOGGING === 'true',  
  entities: [],  
  migrations: [],  
});
```

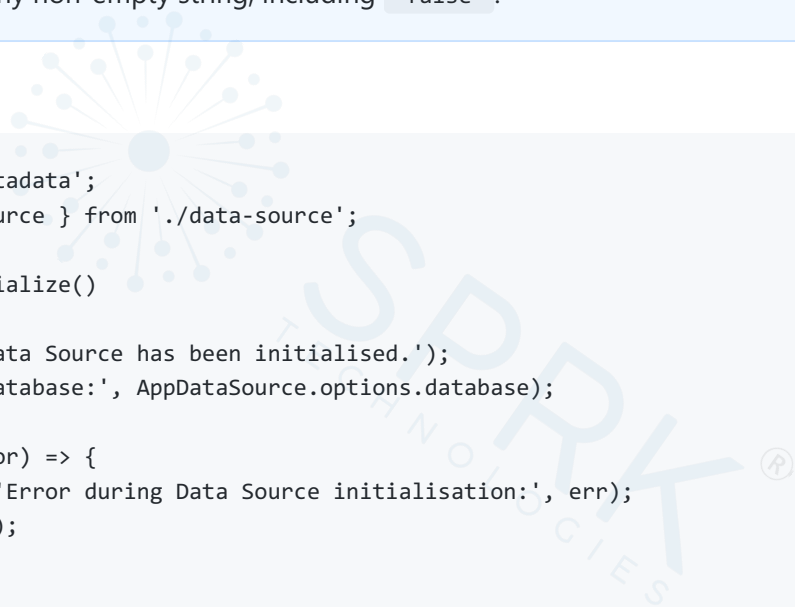
Note — Type assertion on `type`: TypeORM's `type` option expects a specific string literal union (`'better-sqlite3'` | `'postgres'` | `'mysql'` | ...), not a plain `string` . Since `process.env.DB_TYPE` returns `string` | `undefined` , TypeScript cannot automatically narrow it to the correct literal. The `as 'better-sqlite3'` cast tells the compiler which literal type to expect. In a real multi-database project, you would use a validation function here rather than a cast.

Note — Boolean environment variables: All environment variable values are strings. `process.env.DB_SYNCHRONIZE` returns the string `"true"` , not the boolean `true` . Comparing with `=== 'true'` is the idiomatic conversion. Never write `Boolean(process.env.DB_SYNCHRONIZE)` — it returns `true` for any non-empty string, including `"false"` .

src/app.ts

```
import 'reflect-metadata';
import { AppDataSource } from './data-source';

AppDataSource.initialize()
  .then(() => {
    console.log('Data Source has been initialised.');
```



```
    console.log('Database:', AppDataSource.options.database);
  })
  .catch((err: Error) => {
    console.error('Error during Data Source initialisation:', err);
    process.exit(1);
  });
```

npm run dev

Output:

```
query: BEGIN TRANSACTION
query: COMMIT
Data Source has been initialised.
Database: database.db
```

The connection is established identically to Stage 1, but no credentials exist in the source code. Changing the database requires editing `.env` only — the TypeScript files are untouched.

DataSource Configuration for PostgreSQL

PostgreSQL and MySQL require additional connection options that SQLite does not — specifically, a host, port, username, and password. The DataSource options differ by `type`; TypeORM's TypeScript overloads enforce which options are valid for each database type.

This section is reference material. Your practicals use SQLite — come back here when deploying to a PostgreSQL environment.

`.env (PostgreSQL)`

```
DB_TYPE=postgres
DB_HOST=localhost
DB_PORT=5432
DB_USERNAME=myuser
DB_PASSWORD=mypassword
DB_DATABASE=myapp_dev
DB_SYNCHRONIZE=false
DB_LOGGING=true
```

`src/data-source.ts (PostgreSQL)`

```
import 'reflect-metadata';
import { DataSource } from 'typeorm';
import * as dotenv from 'dotenv';

dotenv.config();

export const AppDataSource = new DataSource({
  type: 'postgres',
  host: process.env.DB_HOST ?? 'localhost',
  port: parseInt(process.env.DB_PORT ?? '5432', 10),
  username: process.env.DB_USERNAME ?? 'postgres',
  password: process.env.DB_PASSWORD ?? '',
  database: process.env.DB_DATABASE ?? 'myapp',
  synchronize: process.env.DB_SYNCHRONIZE === 'true',
  logging: process.env.DB_LOGGING === 'true',
  entities: [],
});
```

```
migrations: [],  
ssl: process.env.DB_SSL === 'true' ? { rejectUnauthorized: false } : false,  
});
```

Note — `parseInt` for port: `process.env.DB_PORT` is a string. The `DataSource` `port` option expects a number. Always parse it with `parseInt(value, 10)` — the radix argument (`10`) ensures decimal parsing regardless of leading zeros.

Note — SSL in production: Managed PostgreSQL services (Heroku, Render, Railway, Supabase) typically require SSL connections. The `ssl` option in the `DataSource` handles this.

`rejectUnauthorized: false` skips certificate validation — acceptable for managed services with trusted certificates, but review the documentation of your specific provider for the correct SSL configuration.

Registering Entities: Class References vs Glob Patterns

The `entities` array tells TypeORM which classes to treat as database entities. There are two ways to populate it: explicit class references, or a glob path pattern.

Explicit Class References

Import each entity class and include it in the array. This is the clearest approach — TypeScript knows the exact types and will error if an import path is wrong.

`src/data-source.ts` (explicit – shown here for reference)

```
import { User } from './entity/User';  
import { Post } from './entity/Post';  
  
export const AppDataSource = new DataSource({  
  // ...other options  
  entities: [User, Post],  
});
```

Glob Pattern

Pass a file path glob and TypeORM discovers entity files automatically. This avoids updating the DataSource every time a new entity is added.

```
src/data-source.ts (glob pattern)
```

```
export const AppDataSource = new DataSource({
  // ...other options
  entities: [__dirname + '/entity/**/*.{js,ts}'],
});
```

Important — Glob patterns and ts-node : When running with `ts-node` in development, entity files are `.ts`. After compiling with `tsc`, they become `.js` in the `dist/` folder. The glob `**/*.{js,ts}` handles both. Alternatively, keep development and production DataSource configs separate — a common pattern for larger projects. For practicals, explicit class references are used: they are explicit, they catch typos at compile time, and the entity count remains small.

Initialising Once and Sharing the Connection

`AppDataSource.initialize()` is asynchronous — it opens the connection pool and performs TypeORM's internal setup. It must complete before any database operation can run. Calling it more than once throws an error. The pattern used throughout this module is to initialise in the application entry point and export the already-initialised DataSource for use elsewhere.

Stage 3 — Entry Point Pattern with Express

The Express server is created inside the `.then()` callback of `initialize()`. This guarantees the database is ready before the server begins accepting requests.

```
src/app.ts
```

```
import 'reflect-metadata';
import express, { Request, Response } from 'express';
import { AppDataSource } from './data-source';
```

```
AppDataSource.initialize()
  .then(() => {
    console.log('Database connected.');
```



```
    const app = express();
    app.use(express.json());
```



```
    app.get('/health', (req: Request, res: Response) => {
      res.json({
        status: 'ok',
        database: AppDataSource.options.database,
        isInitialised: AppDataSource.isInitialized,
      });
    });
```



```
    app.listen(3000, () => {
      console.log('Server running on http://localhost:3000');
    });
  })
  .catch((err: Error) => {
    console.error('Failed to connect to database:', err.message);
    process.exit(1);
  });
```

```
npm run dev
```

Output:

```
query: BEGIN TRANSACTION
query: COMMIT
Database connected.
Server running on http://localhost:3000
```

Now test the health endpoint:

```
curl http://localhost:3000/health
```

Output:

```
{
  "status": "ok",
  "database": "database.db",
  "isInitialised": true
}
```

`AppDataSource.isInitialized` is a TypeORM property that returns `true` once `initialize()` has resolved successfully. It is useful in health check endpoints and middleware that should only proceed if the database is available.

Using AppDataSource in Other Modules

Because `AppDataSource` is exported from `data-source.ts` and `initialize()` is called once in `app.ts`, any other module that imports `AppDataSource` receives the same already-initialised instance. Node.js module caching ensures the singleton — the module is evaluated once and the same exported object is returned on every subsequent `require / import`.

`src/routes/user.routes.ts` (example – entities covered in Section 11)

```
import { Router } from 'express';
import { AppDataSource } from '../data-source';
// Entity imports will be added in Section 11

const router = Router();

router.get('/db-status', (req, res) => {
  res.json({ connected: AppDataSource.isInitialized });
});

export default router;
```

Output:

```
GET /db-status → { "connected": true }
```

Understanding TypeORM Logging

TypeORM's `logging` option controls how much information is printed to the console. During development, seeing every SQL statement TypeORM generates is essential — it confirms the ORM is doing what you expect and reveals unexpected queries (N+1 problems, unnecessary joins, etc.).

Value	What is logged	Use when
<code>false</code>	Nothing	Production (use a dedicated logger instead)
<code>true</code>	All queries, errors, and schema sync statements	Active development — maximum visibility
<code>['query']</code>	Only SELECT / INSERT / UPDATE / DELETE statements	When schema sync noise is distracting
<code>['error']</code>	Only failed queries	Staging environments
<code>['query', 'error', 'warn']</code>	Queries, errors, and deprecation warnings	Detailed debugging without schema sync noise
<code>'all'</code>	Everything including slow query warnings	Performance investigation

With `logging: true` active, here is what a typical startup sequence with one entity looks like (you will see this properly in Section 11, but recognising the format now is useful):

Output:

```
query: BEGIN TRANSACTION
query: SELECT * FROM "sqlite_master" WHERE "type" = 'table' AND "name" IN ('user')
query: CREATE TABLE "user" ("id" integer PRIMARY KEY AUTOINCREMENT NOT NULL, "name" varchar NOT NULL, "email" varchar NOT NULL UNIQUE)
query: COMMIT
Database connected.
Server running on http://localhost:3000
```


The `SELECT * FROM sqlite_master` is TypeORM checking whether the `user` table already exists before deciding to create it. The `CREATE TABLE` is the synchronisation step. On subsequent starts, if the schema matches, no DDL is emitted — only the transaction open and commit.

Summary

Concept	Key Point
Packages to install	<code>typeorm</code> + <code>reflect-metadata</code> + a database driver (<code>better-sqlite3</code> for practicals, <code>pg</code> for PostgreSQL)
<code>reflect-metadata</code> import	Must be the first import in the application entry point — a side-effect import: <code>import 'reflect-metadata';</code>
<code>experimentalDecorators</code>	tsconfig flag required to enable TypeScript decorator syntax (<code>@Entity()</code> , <code>@Column()</code> , etc.)
<code>emitDecoratorMetadata</code>	tsconfig flag required so TypeORM can read TypeScript type metadata at runtime via <code>reflect-metadata</code>
Definite assignment (<code>!</code>)	Required on entity properties under <code>strict: true</code> — tells TypeScript the ORM will initialise them: <code>id!: number;</code>
<code>DataSource</code>	The central TypeORM configuration object. Constructed once, exported as a singleton, initialised asynchronously before the server starts.
<code>ormconfig.json</code>	Deprecated pre-0.3 pattern. Do not use in new projects — construct a <code>DataSource</code> directly instead.

<code>synchronize: true</code>	Auto-creates and alters tables on startup. Development only — can silently destroy data in production. Use migrations (Section 12) instead.
Environment variables	Credentials stored in <code>.env</code> , loaded by <code>dotenv.config()</code> , read via <code>process.env</code> . Never commit <code>.env</code> to git.
Boolean env vars	All env values are strings — convert with <code>=== 'true'</code> , never with <code>Boolean()</code> which returns <code>true</code> for any non-empty string.
Initialise-then-listen pattern	Start the Express server inside the <code>.then()</code> of <code>initialize()</code> to guarantee the DB is ready before accepting requests.
Entity registration	Pass class references (<code>entities: [User, Post]</code>) or a glob (<code>'src/entity/**/*.{js,ts}'</code>). Explicit references are preferred in small projects.

Interview Questions

1. Why is `reflect-metadata` a separate package and not built into TypeScript?

Expected answer: Decorator metadata reflection is a proposed ECMAScript feature that has not yet been standardised. The `reflect-metadata` package is a polyfill for the `Reflect` API that enables runtime introspection of TypeScript-emitted type information. TypeScript relies on it to read the TypeScript type of each decorated property — for example, to know that a property typed as `number` should map to an `INTEGER` column. Once the `Reflect` API becomes a web standard, a polyfill will no longer be needed.

2. What happens if you call `AppDataSource.initialize()` twice?

Expected answer: TypeScript throws an error: *"DataSource is already connected, cannot be connected again."* A `DataSource` can only be initialised once per instance. The solution is to

create the `DataSource` as a module-level export and initialise it once in the entry point — Node.js module caching ensures every import of the module receives the same already-initialised instance.

3. A developer sets `synchronize: true` on the production server to avoid running migrations. What is the risk?

Expected answer: On every application restart, TypeORM compares the live database schema against the entity definitions and issues DDL statements to reconcile any differences. If a developer renames a property in an entity, TypeORM drops the old column and creates a new one — permanently deleting all data in that column. There is no confirmation prompt and no rollback. Migrations (Section 12) are explicit, version-controlled, and reversible; `synchronize` is implicit and irreversible.

4. Why does `Boolean(process.env.MY_FLAG)` not work correctly for boolean environment variables?

Expected answer: `Boolean()` converts any truthy value to `true`. Any non-empty string is truthy — including the string `"false"`. So `Boolean(process.env.MY_FLAG)` returns `true` when `MY_FLAG=false` is set in the environment, which is the opposite of the intended behaviour. The correct idiom is `process.env.MY_FLAG === 'true'`, which returns `false` for any value other than the exact string `"true"`.

5. What is the purpose of the `emitDecoratorMetadata` `tsconfig` option, and what happens at runtime if it is missing?

Expected answer: `emitDecoratorMetadata` causes the TypeScript compiler to inject type metadata into the compiled JavaScript output — specifically, the reflected type of each decorated class property. TypeORM reads this metadata at runtime via `reflect-metadata` to determine column types automatically (e.g., a `number` property becomes an `INTEGER` column without needing an explicit `type: 'int'` in the `@Column` decorator). If the flag is missing, TypeORM falls back to `object` for all types, typically resulting in columns defaulting to `TEXT` or throwing a metadata error.

6. You have a team of five developers. How do you manage database configuration so each developer can use a different local database without overwriting each other's settings?

Expected answer: Each developer creates their own local `.env` file, which is listed in `.gitignore` and never committed. A `.env.example` file with placeholder values is

committed to the repository as a template. When a new developer clones the project, they copy `.env.example` to `.env` and fill in their local credentials. CI/CD pipelines inject the appropriate credentials as environment variables in the pipeline configuration, not from a file.

7. Explain the "initialise-then-listen" pattern and why the server start is deferred until after the database connection is established.

Expected answer: `AppDataSource.initialize()` is asynchronous. If the Express server starts listening before the `DataSource` is initialised, incoming requests that attempt database operations will fail immediately — the repository methods cannot execute queries against an uninitialised `DataSource`. By starting the server inside the `.then()` callback of `initialize()`, the application guarantees the database is ready before it accepts any traffic. If initialisation fails, the `.catch()` handler logs the error and exits the process, preventing a half-functional server from starting.

8. What is the difference between passing entity class references and a glob pattern to the `entities` array? What is the trade-off?

Expected answer: Class references (`entities: [User, Post]`) require an explicit import for each entity — TypeScript validates the import path at compile time, so a missing entity or mistyped path produces a build error. A glob pattern (`'src/entity/**/*.ts'`) discovers entities automatically at runtime — no manual update is needed when adding new entities, but a misconfigured glob silently loads zero entities with no compile-time warning. For small projects, explicit references are preferable for their clarity; for large projects with many entities, the glob pattern reduces maintenance overhead.

Practical Assignment — Wiring Up the DataSource

Objective: Build the TypeORM project foundation from scratch — packages, TypeScript configuration, environment-driven `DataSource`, and a verified connection — that all subsequent practicals in Sections 10 through 14 will build on. Take this setup seriously: a misconfigured `DataSource` produces confusing errors in later sections.

Steps

1. Create a new project directory and initialise it:

```
mkdir typeorm-project && cd typeorm-project
npm init -y
```

2. Install dependencies:

```
npm install typeorm reflect-metadata better-sqlite3 dotenv express
npm install --save-dev typescript ts-node nodemon @types/node @types/express @types/better-sqlite3
```

3. Create `tsconfig.json` with **all five required options**: `strict`, `esModuleInterop`, `experimentalDecorators`, `emitDecoratorMetadata`, and the `outDir` / `rootDir` pair. Verify the file is syntactically valid by running `npx tsc --noEmit` — it should produce no errors on an empty `src/`.

4. Add the following scripts to `package.json`: `"dev": "nodemon --watch src --ext ts --exec ts-node src/app.ts"`, `"build": "tsc"`, `"start": "node dist/app.js"`.

5. Create `.env` with the four SQLite variables shown in the notes. Create `.gitignore` listing `node_modules/`, `dist/`, `*.db`, and `.env`. Create `.env.example` as a committed template with the same keys and placeholder values.

6. Create `src/data-source.ts` using the environment-variable-driven `DataSource` from Stage 2. Leave `entities: []` for now.

7. Create `src/app.ts` using the initialise-then-listen pattern from Stage 3. Add a `GET /health` route that returns `{ status, database, initialised }`.

8. Run `npm run dev` and verify the output matches the expected output below. Then test the health endpoint with `curl http://localhost:3000/health` or your browser.

9. Stop the server. Verify that `database.db` was created in the project root. Open it with a SQLite viewer (DB Browser for SQLite is free and cross-platform) or run:

```
npx ts-node -e "const {AppDataSource}=require('./src/data-source');
AppDataSource.initialize().then(()=>console.log(AppDataSource.driver.database))"
```

You should see the database path printed.

10. Deliberately break the configuration — set `DB_SYNCHRONIZE=false_typo` in `.env` — and confirm that `synchronize` evaluates to `false` (not an error, just the wrong value). Restore it. This exercise reinforces why boolean env var conversion requires strict string comparison.

Expected Output

Output:

```
query: BEGIN TRANSACTION
query: COMMIT
Database connected.
Server running on http://localhost:3000

--- curl http://localhost:3000/health ---
{
  "status": "ok",
  "database": "database.db",
  "isInitialised": true
}

--- project root after first run ---
typeorm-project/
├─ database.db      ← created by SQLite on first connection
├─ .env
├─ .env.example
├─ .gitignore
├─ package.json
├─ tsconfig.json
└─ src/
    ├─ app.ts
    └─ data-source.ts
```

Bonus Challenge

Add a second Express route `GET /config` that returns a JSON response showing which `type` and `database` the `DataSource` is currently using — read from `AppDataSource.options`. Add a guard: if `AppDataSource.isInitialized` is `false`, return a `503 Service Unavailable` response instead. This is a minimal but realistic health-check pattern used in production services.

Notes on content additions/changes from syllabus: The syllabus lists three bullet points for 10-02. This document adds: (1) an explicit package installation table distinguishing `typeorm`, `reflect-metadata`, and the database driver — the syllabus says "installing TypeORM" but the three-package requirement is a frequent source of confusion and deserves its own table; (2) the `emitDecoratorMetadata` explanation and the definite assignment assertion `!` pattern — both are undocumented pain points that students hit immediately and without prior warning; (3) the deprecated `ormconfig.json` note, because students following older tutorials will encounter it and need to understand why the notes use a different approach; (4) the PostgreSQL



DataSource configuration as reference material, since the practicals use SQLite but students need the PostgreSQL form for any real deployment; (5) the logging options table — "environment variables" is listed in the syllabus, but understanding what the logs mean is equally critical for debugging during the subsequent sections.

